

Algoritmos y Estructuras de Datos en Python

Fundamentos, Análisis y Aplicaciones para la Inteligencia Artificial.

Un recorrido visual estructurado por módulos.

Módulo 1 | La Anatomía de un Algoritmo



Independiente del lenguaje.

El algoritmo es la infraestructura conceptual; el código en Python es simplemente su implementación técnica.

Contenedores Nativos de Python



Listas (list)

Ordenadas y mutables.
Uso: Secuencias de datos dinámicas y vectores de características.



Tuplas (tuple)

Ordenadas e inmutables.
Uso: Datos fijos, coordenadas, protección contra modificaciones.



Conjuntos (set)

No ordenados, elementos únicos. **Uso:** Eliminar duplicados, validaciones de pertenencia, intersecciones.



Diccionarios (dict)

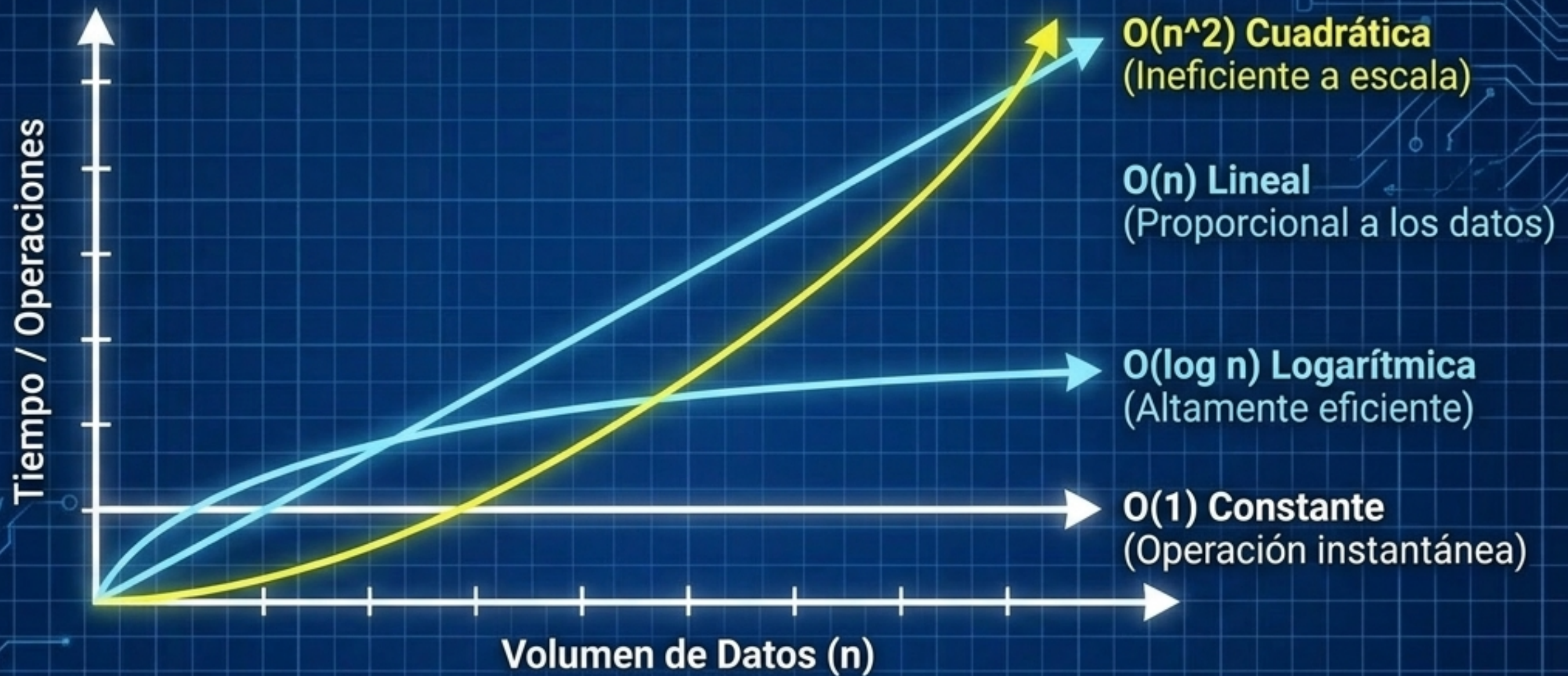
Pares Clave-Valor. **Uso:** Búsquedas ultrarrápidas, representación de metadatos o instancias con atributos.

Tipos de Datos Abstractos (TDA) y POO



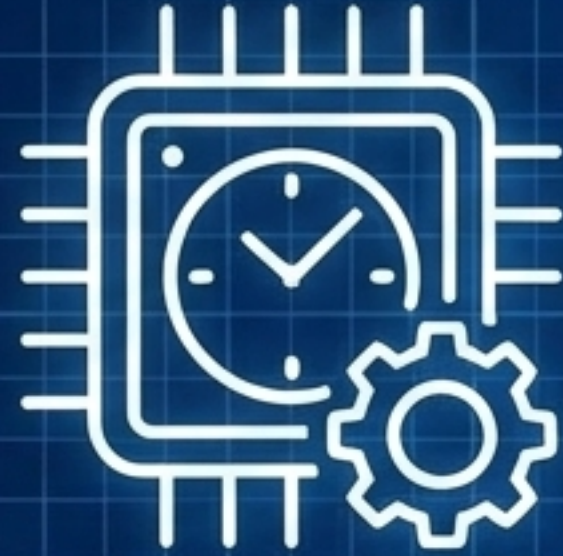
Un TDA separa el qué hace una estructura del cómo se implementa. En Python, utilizamos el encapsulamiento de la Programación Orientada a Objetos (Clases) para modelar y proteger esta arquitectura.

Módulo 2 | Complejidad Algorítmica (Notación Big-O)



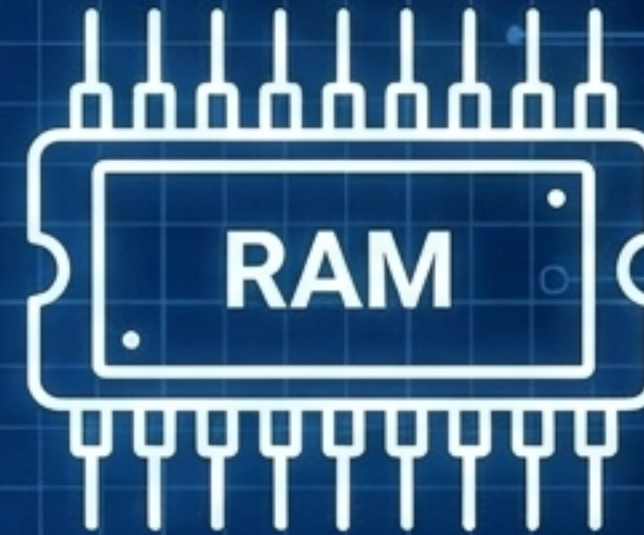
No medimos el tiempo en segundos, medimos cómo crece el **esfuerzo computacional** a medida que aumenta el volumen de datos.

El Trade-off: Tiempo vs. Espacio



Complejidad Temporal

¿Cuánto tiempo (pasos/iteraciones) requiere el algoritmo para completarse en el peor de los casos?

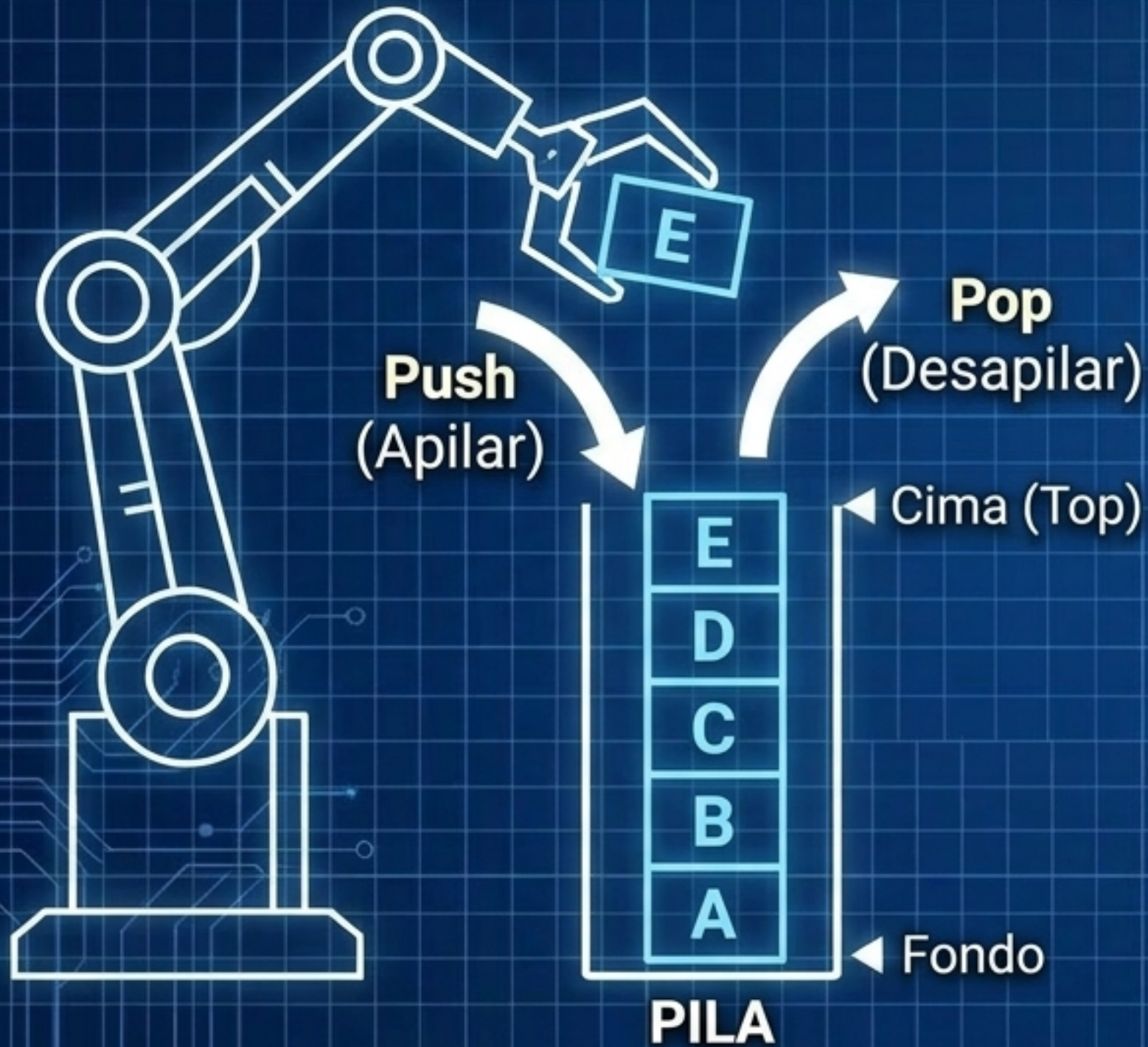


Complejidad Espacial

¿Cuánta memoria extra (variables, estructuras auxiliares) requiere el algoritmo para ejecutarse?

Trade-off: En Inteligencia Artificial, el diseño de algoritmos a menudo requiere sacrificar memoria para ganar velocidad, o viceversa.

Pilas (Stacks) y el Principio LIFO

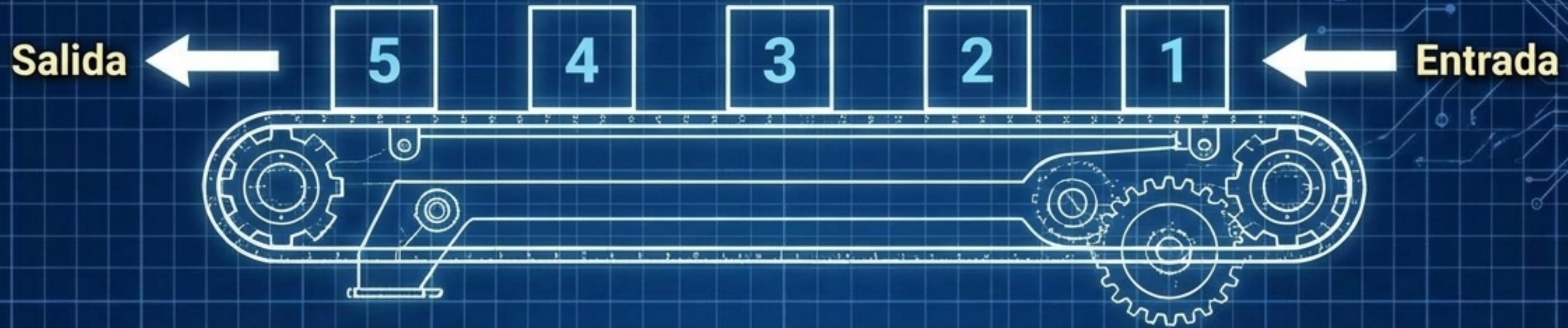


Principio LIFO (Last In, First Out) - El último elemento en entrar es el primero en salir.

Rendimiento: Las operaciones Push y Pop garantizan una complejidad temporal de $O(1)$.

Casos de Uso: Ideal para modelar Recursividad, Retroceso (Backtracking en laberintos), e historial de acciones.

Colas (Queues) y el Principio FIFO



Deque (Doble Terminación)

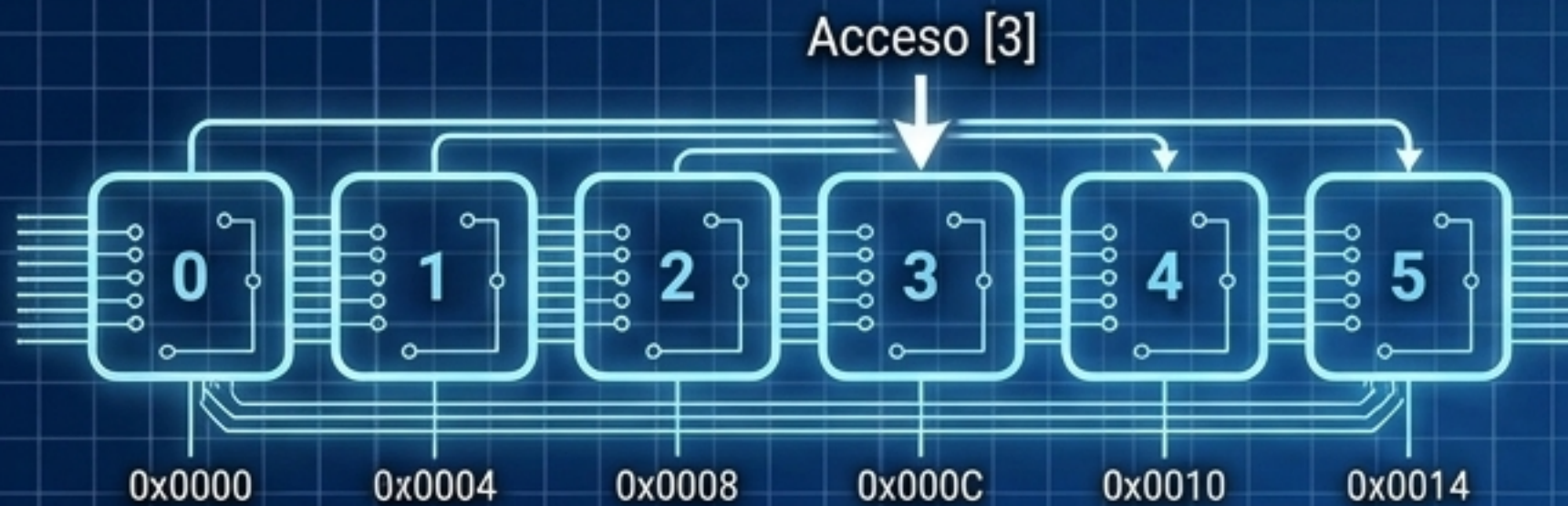
Permite operaciones eficientes de entrada y salida en ambos extremos.

Cola de Prioridad (Priority Queue)

La salida se basa en la importancia (peso/costo) del elemento, no en su orden de llegada.

Casos de Uso: Simulaciones, algoritmos de Búsqueda por Niveles (BFS), y atención secuencial de tareas.

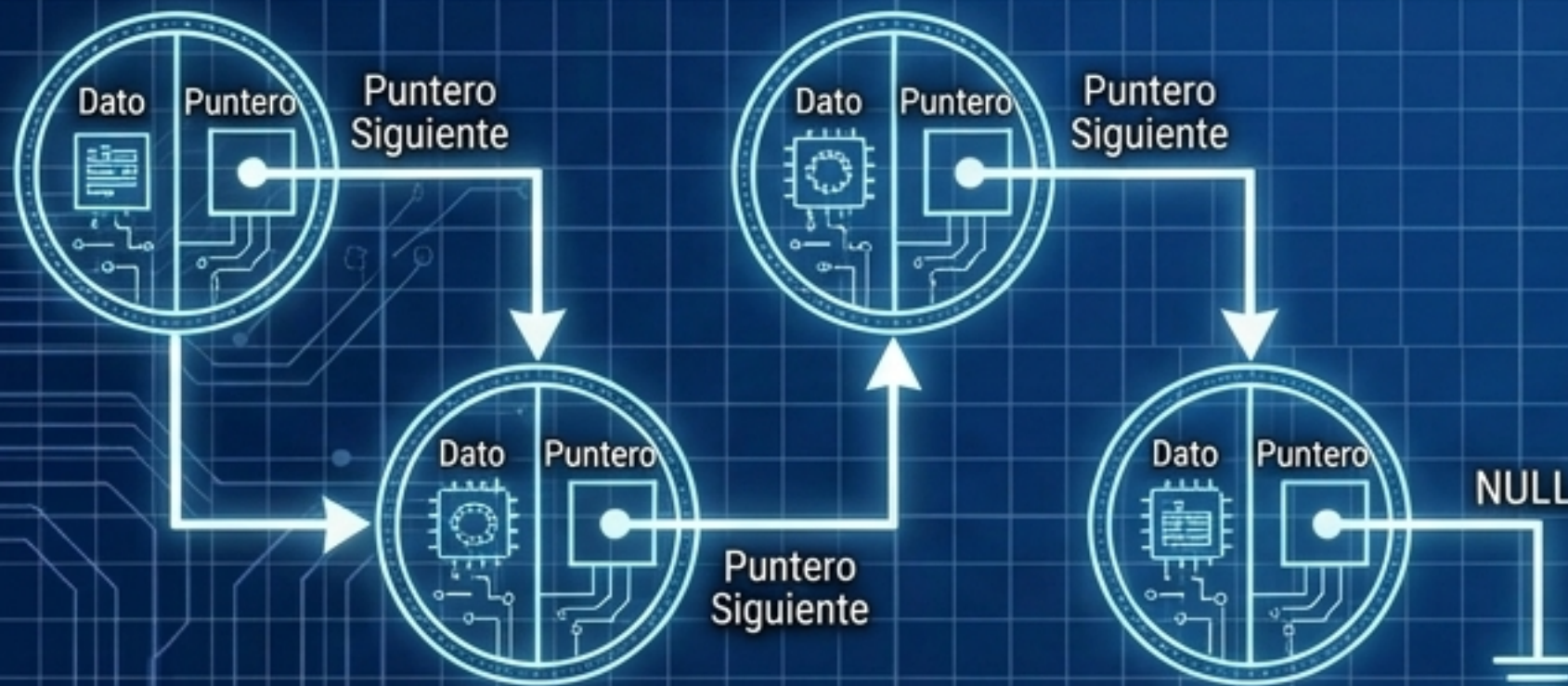
Gestión de Memoria: Arreglos vs. Listas Enlazadas



Arreglos / Listas Dinámicas: Memoria contigua.

Ventaja: Acceso aleatorio instantáneo $O(1)$.

Desventaja: Inserción en el medio es lenta $O(n)$ por desplazamiento.

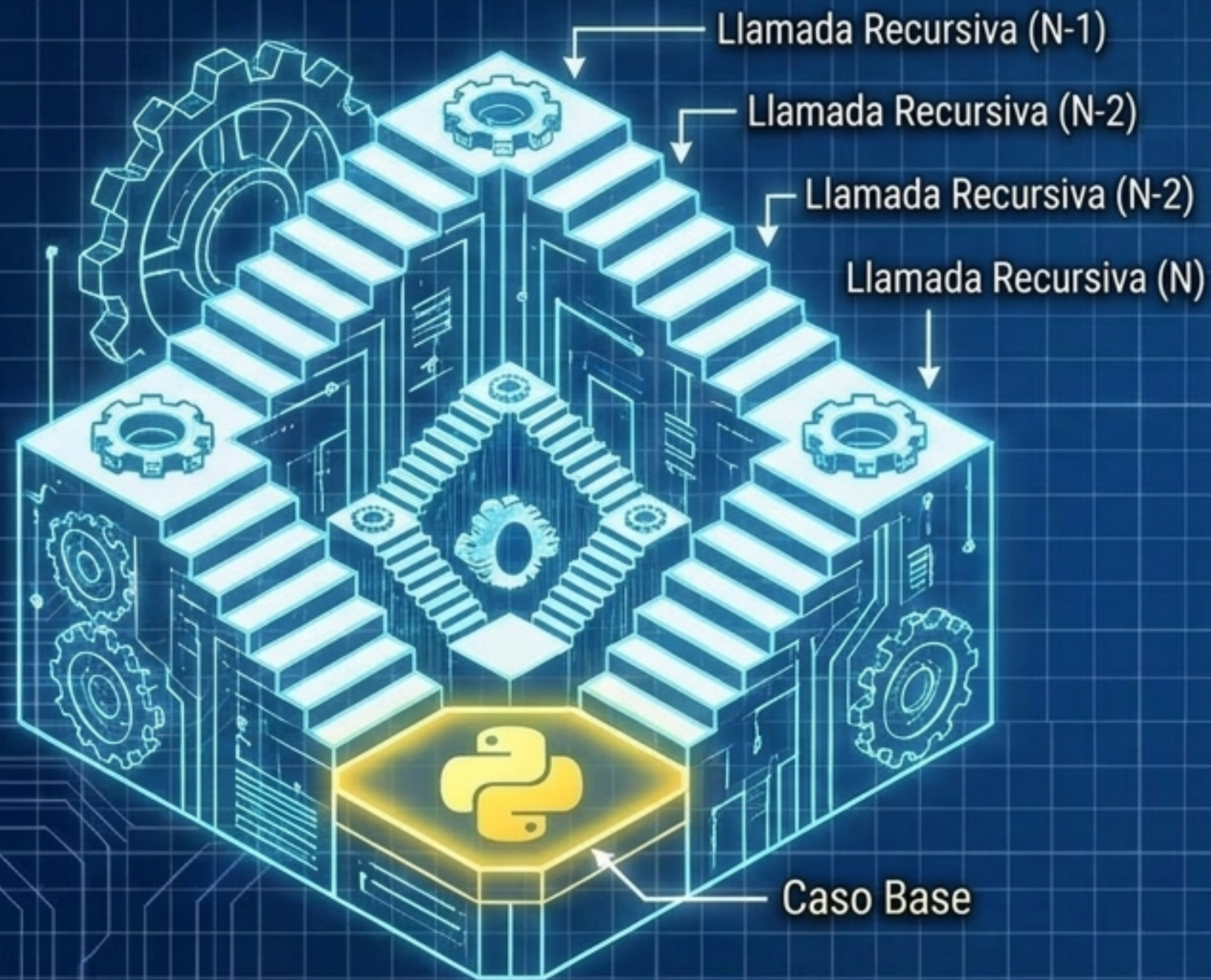


Listas Enlazadas: Nodos dispersos (Dato + Puntero).

Ventaja: Inserción/eliminación rápida $O(1)$.

Desventaja: Acceso secuencial lento $O(n)$.

Módulo 3 | La Naturaleza de la Recursividad



Concepto: Una función que se llama a sí misma para dividir un problema complejo en subproblemas idénticos más pequeños.

Anatomía de la Recursividad:

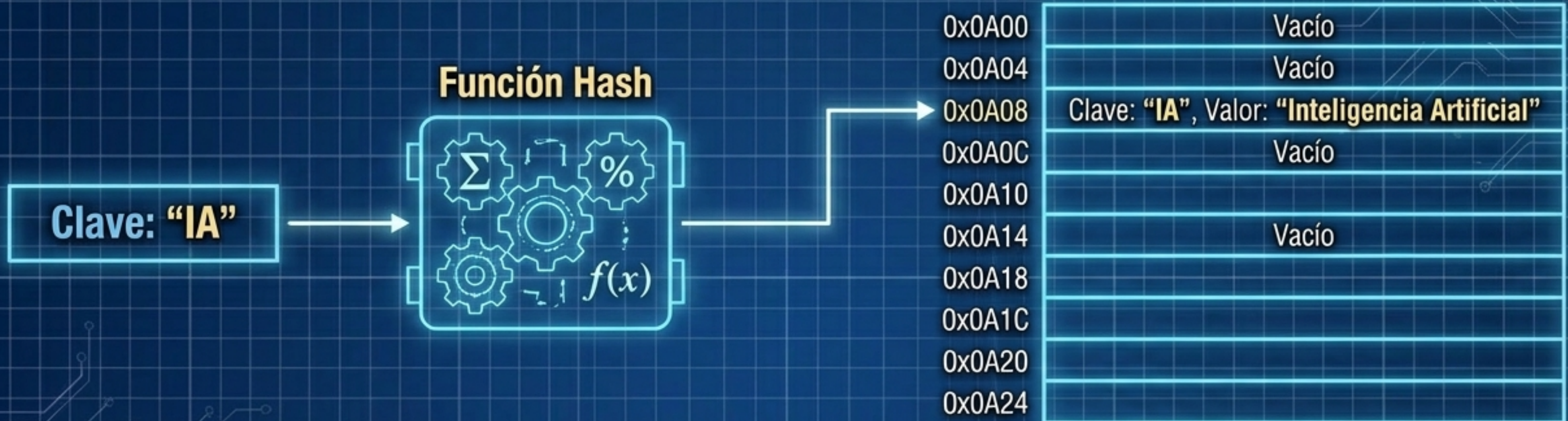
1. **Caso Base:** La condición de parada absoluta. (Evita el Stack Overflow).
2. **Llamada Recursiva:** La instrucción que acerca el problema un paso más hacia el caso base.

Ejemplo Clásico: El cálculo de Fibonacci o Factoriales.

Algoritmos de Ordenamiento



Tablas Hash y la Búsqueda Instantánea



Mecanismo: Utiliza una función matemática (hash) para transformar una Clave en una Dirección de memoria directa.

Eficiencia: Permite acceso, inserción y búsqueda en tiempo promedio de $O(1)$.

Implementación: Arquitectura subyacente que hace que los dict y set de Python sean formidables.

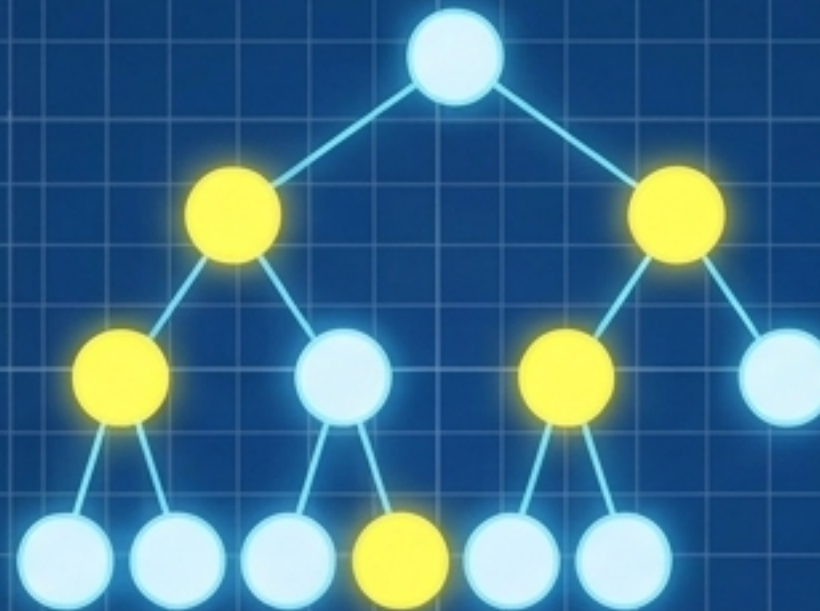
Estructuras Jerárquicas: Árboles de Búsqueda

Árbol AVL



Estrictamente balanceado (diferencia de altura ≤ 1). Maximiza la velocidad en memoria RAM. Requiere múltiples rotaciones al insertar.

Árbol Rojo-Negro



Balanceo aproximado/relajado mediante reglas de color. Menos rotaciones al insertar. Utilizado como base para sets y maps.

Árbol B (Multivía)



Diseño achatado con múltiples hijos por nodo. Optimizado específicamente para minimizar accesos a disco duro y bases de datos.

Grafos y Estrategias de Exploración

Redes compuestas por Vértices (estados/nodos) y Aristas (transiciones).

BFS (Búsqueda en Anchura)



Explora por niveles. Implementado con una Cola (FIFO).
Estrategia IA: Ideal para encontrar el camino más corto.

DFS (Búsqueda en Profundidad)



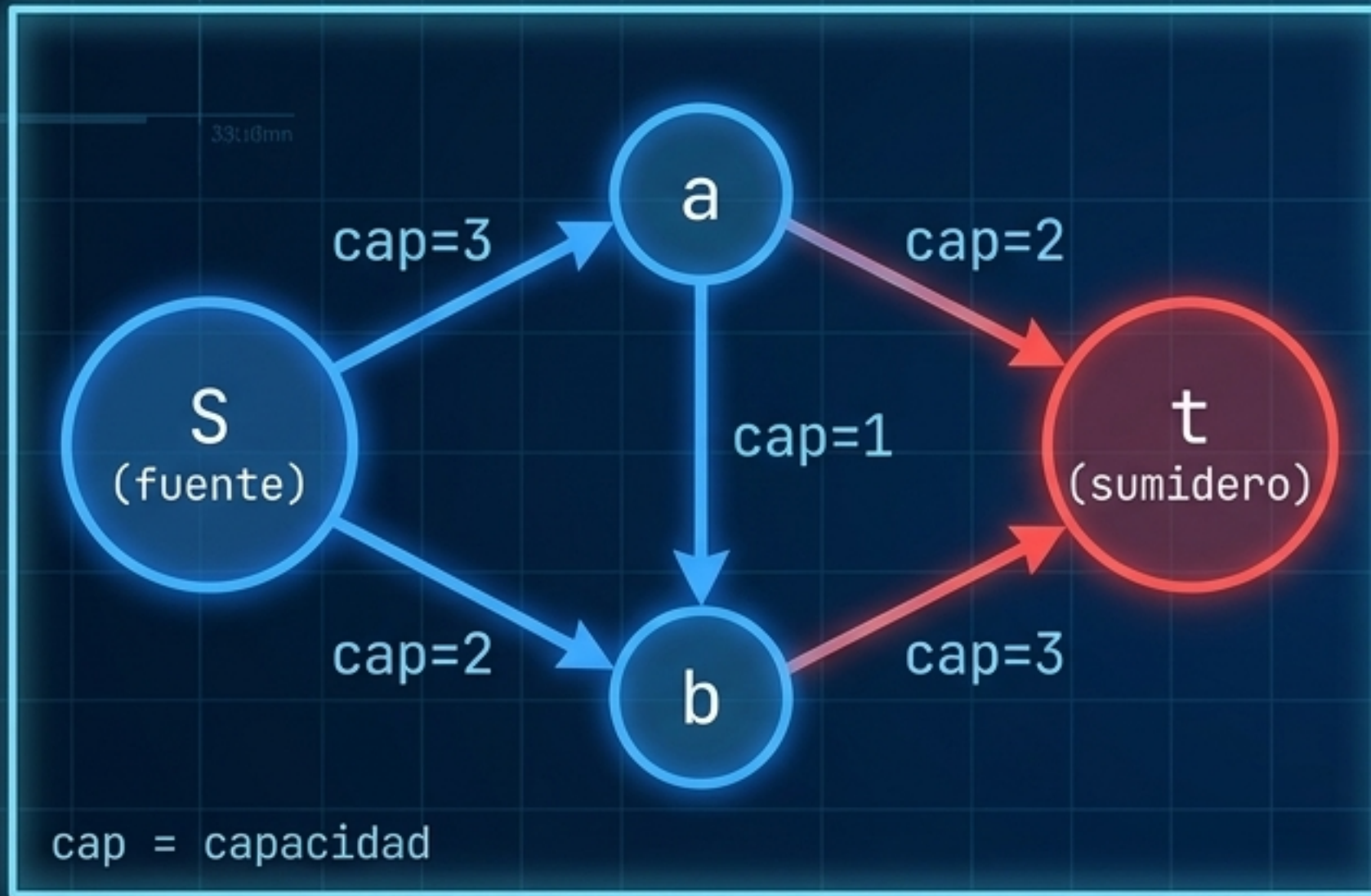
Explora una rama hasta el final antes de retroceder (Backtracking). Implementado con una Pila (LIFO).
Estrategia IA: Ideal para explorar todas las rutas o laberintos.

DWG. NO.

SCALE: NTS

ENGINEERING DIVISION

Redes de Flujo y Optimización



Concepto:

Concepto: Un dígrafo donde las aristas representan tuberías con una capacidad máxima de transmisión.

Reglas Fundamentales:

1. **Restricción de capacidad:** El flujo no puede superar el límite de la arista.
2. **Conservación de flujo:** Lo que entra a un nodo intermedio debe salir exactamente igual.

Aplicación en IA: Modelado fundamental para optimización de recursos, ruteo de telecomunicaciones, redes de transporte y asignación computacional.